

5 mar 2026

Bootcamp Desarrollo Sesión

Invitados Iztac Amaya Arturo Durán cuahutli.lopez@disrupttglobal.com
oscar.lopez@disrupttglobal.com nora.simental@disrupttglobal.com
carlos.espinosa@disrupttglobal.com jorge.lopez@disrupttglobal.com

Archivos adjuntos 📎 Bootcamp Desarrollo Sesión

🔴 Bootcamp Desarrollo Sesión - 2026/03/05 09:30 CST - Recording

Registros de la reunión 🔴 Grabación 🔴 Grabación 2

Resumen

Iztac Amaya, Jorge Nicolas Moctezuma Arellano, Arturo Durán, Carlos Espinosa, Ollin Palafox, Nora Simental, y Oscar Herrera revisaron el estado de las prácticas de Client Script y User Event, confirmando que Carlos Espinosa completó ambos y que Jorge Nicolas Moctezuma Arellano pudo hacer funcionar el User Event, mientras que Ollin Palafox reportó problemas con el manifiesto SDF en Visual Code y Nora Simental y Oscar Herrera tenían pendientes. Iztac Amaya introdujo Sweetlets y Restlets, demostrando su funcionalidad para generar pantallas personalizadas y para permitir que NetSuite reciba y envíe información a otros sistemas mediante peticiones HTTP (GET y POST), con énfasis en la configuración de autenticación OAuth 1.0 para el Restlet. Se asignó una práctica final enfocada en el desarrollo de un Sweetlet para actualizar registros en NetSuite, combinando los módulos 'record' y 'search', con la instrucción de utilizar la misma rama de Git.

Detalles

- **Revisión de las prácticas de Client Script y User Event:** Iztac Amaya preguntó sobre las prácticas pendientes de Client Script y User Event, a lo que Jorge

Nicolas Moctezuma Arellano respondió que no pudo ver el Client Script en funcionamiento, aunque logró desplegarlo. Arturo Durán interrumpió brevemente para preguntar sobre un problema de archivos en el correo electrónico mencionado previamente.

- **Estado de desarrollo y desafíos con User Event:** Iztac Amaya confirmó que Jorge Nicolas Moctezuma Arellano completó el Client Script, pero le faltó ver el User Event funcionando. Carlos Espinosa indicó que completó ambos desarrollos y los subió a Git.
- **Dificultades con Git y el manifiesto SDF:** Ollin Palafox, identificado como "López" en el sistema junto con otros, informó haber completado ambos scripts (Client Script y User Event). Sin embargo, Ollin Palafox tuvo problemas para poner los scripts en modo testing o producción y reportó que Visual Code dejó de reconocer el manifiesto al intentar hacer un `*commit*`. Jorge Nicolas Moctezuma Arellano comentó que logró abrir el repositorio de GitHub en su cuenta personal.
- **Actualización del estado de los scripts y necesidad de prácticas:** Se aclaró el estado de los scripts con Jorge Nicolas Moctezuma Arellano, quien indicó que el User Event ya funcionaba correctamente, pero aún no podía ver el Client Script en NetSuite. Nora Simental informó que no había completado el User Event, y Oscar Herrera se quedó atascado con el Client Script, aunque subió el User Event. Iztac Amaya enfatizó la importancia de practicar y familiarizarse con Git, el proyecto SDF, Visual Code, y NetSuite, sugiriendo que las dudas de Git podrían abordarse en un "miniurso" posterior.
- **Introducción a los Sweetlets y su funcionalidad:** Iztac Amaya introdujo los Sweetlets, describiéndolos como scripts fundamentales en el desarrollo de NetSuite para generar pantallas personalizadas mediante código, utilizando peticiones HTTP de tipo GET o POST. Un Sweetlet puede generar formularios con el aspecto estándar de NetSuite, incluyendo secciones, campos (texto, selección, moneda), pestañas y botones, o se pueden crear Sweetlets avanzados con un diseño totalmente diferente utilizando `*frameworks*` de desarrollo web.
- **Estructura y ejecución de un Sweetlet:** Se explicó que una petición GET se usa para renderizar la página o el formulario, mientras que una petición POST se activa cuando el usuario envía datos (por ejemplo, al hacer clic en un botón de "enviar"). Iztac Amaya demostró la estructura de un Sweetlet que utiliza el módulo ``service`` para construir la parte visual, y mostró cómo un Sweetlet normal tiene un único punto de entrada: el ``onRequest``, donde se valida si la

petición es GET o POST. El código de demostración mostraba cómo pintar un formulario simple con campos de texto, fecha, moneda, y una sublista en la parte GET, y cómo procesar los datos ingresados en la parte POST.

- **Despliegue de Sweetlet y su accesibilidad:** El Sweetlet debe pasar por tres pasos: el archivo de script (`script file`), el registro de script (`script record`), y el despliegue (`deployment`). Iztac Amaya demostró que una vez desplegado, el Sweetlet genera una URL para acceder al formulario. También se mostró la opción de vincular la URL del Sweetlet a un centro de NetSuite (por ejemplo, el Centro Clásico) para facilitar el acceso a los usuarios. Además, se mencionó que se pueden hacer Sweetlets accesibles sin iniciar sesión en NetSuite, aunque esto no se recomienda por razones de vulnerabilidad.
- **Funcionalidades avanzadas del Sweetlet (Asistente) y Tipos de Campos:** Iztac Amaya discutió la capacidad de un Sweetlet para crear un "asistente" (wizard) que guía al usuario a través de una serie de pasos para capturar información, lo que permite validaciones y diferentes tipos de campos en cada paso. El módulo `Service Widget` soporta numerosos tipos de campos, incluyendo casilla de verificación (`checkbox`), moneda, fecha/hora, correo electrónico (con validación), carga de archivos, `inline HTML`, imágenes, etiquetas, y selección múltiple.
- **Comparación entre Sweetlets y Custom Records:** Se discutió que, para formularios sencillos, se recomienda usar registros personalizados (*custom records*) en lugar de Sweetlets, ya que los *custom records* se almacenan directamente en NetSuite como si fueran una base de datos. Por el contrario, un Sweetlet proporciona una pantalla temporal que captura datos, ejecuta validaciones, y luego puede, por ejemplo, crear un *custom record* en el proceso de una petición POST.
- **Introducción a los Restlets:** Iztac Amaya presentó los Restlets, indicando que funcionan como "ventanitas" para que NetSuite reciba información de otros sistemas a través de peticiones HTTP (DELETE, GET, POST, PUT). Por ejemplo, una petición GET hacia un Restlet puede ser utilizada por un punto de venta externo para extraer datos de artículos desde NetSuite, como nombres, precios e inventario.
- **Explicación de las Peticiones HTTP y Netsuite:** Iztac Amaya explicó los cuatro tipos de peticiones HTTP: `POST` para crear nuevos registros, `PUT` para actualizar registros existentes, `DELETE` para eliminar datos y `GET` para que

otros sistemas extraigan información de NetSuite. Se confirmó que este mecanismo permite que NetSuite tenga APIs disponibles para otros sistemas, denominadas peticiones de tipo REST, o un 'Restlet'.

- **Proceso de Creación e Implementación de Script Restlet:** Se describió el proceso para generar un nuevo script, incluyendo la selección del módulo `search`. Se enfatizó la importancia de la documentación, aunque Iztac Amaya decidió omitirla por el momento para mantener el código "un poquito más limpio". Los pasos para la implementación de un Restlet incluyen la creación del archivo de script (`script file`), el registro del script (`script record`), y la implementación final (`deployment`), dejando la configuración como accesible para "todos los empleados".
- **Configuración de URLs y Autenticación para Acceso Externo:** Para que la API sea accesible por otros sistemas, se requiere una URL pública. Iztac Amaya detalló el uso de Postman para realizar pruebas unitarias y peticiones a APIs. Se señaló que NetSuite tiene dos métodos de autenticación, OAuth 1.0 y 2.0, optando por configurar el 1.0 para el ejemplo, lo que requiere obtener la **consumer key**, **consumer secret**, **access token** y **token secret**.
- **Generación de Credenciales de Integración y Tokens de Acceso:** La generación de los tokens de autenticación implica crear una nueva integración en NetSuite. Para la autenticación 1.0, se deben activar las características de "autenticación basada en tokens". El proceso concluyó con la generación del **token ID** y **token secret**. Iztac Amaya sugirió que, aunque se puede generar un token para ellos como administradores para fines didácticos, lo ideal es usar la ventana anterior para una gestión más restrictiva de usuario y rol.
- **Configuración de Postman y Pruebas Iniciales de GET:** Se mostró cómo configurar Postman con la URL externa y las credenciales de OAuth 1.0, incluyendo la **account ID** que se obtiene de la información de la compañía en NetSuite. Tras solucionar un error de configuración donde el **Content-Type** debía ser `application/json` en lugar de **JavaScript**, se demostró que la petición `GET` se estaba ejecutando correctamente, retornando los parámetros enviados.
- **Desarrollo del Método GET con Funcionalidad de Búsqueda:** Iztac Amaya demostró la utilidad de los plugins de Chrome para facilitar el desarrollo, como uno que exporta las búsquedas guardadas a código, lo cual se utilizó para crear una función de búsqueda (`search`) en el Restlet. El objetivo de este Restlet es

extraer información de entidades (clientes) y puede recibir parámetros, como un `internal ID`, a través de la URL para filtrar los resultados.

- **Demostración Final de la Funcionalidad de Restlet:** Se demostró el funcionamiento final del Restlet, que utiliza el `internal ID` enviado por Postman para buscar clientes específicos o devolver múltiples resultados si el ID no se envía. Esto ejemplifica cómo otros sistemas pueden usar el Restlet para consultar información en NetSuite, como artículos farmacéuticos o datos de clientes.
- **Asignación de Ejercicios Prácticos (Suitelet y Restlet):** Iztac Amaya propuso dos ejercicios prácticos para que los participantes los trabajen, sugiriendo que se conecten de 11:00 a 1:00 para revisar los resultados. El ejercicio del Suitelet consiste en crear una pantalla con un formulario para seleccionar un cliente y una fecha de nacimiento, y usar el método `POST` para actualizar el cliente. El ejercicio del Restlet implica que la aplicación envíe un ID de cliente y una fecha de nacimiento mediante una petición `POST`.
- **Discusión sobre la programación de la sesión y el uso de módulos:** La sesión fue programada para trabajar en tareas de 9:30 a 11:00. La discusión se centró en la exploración de diferentes formas de utilizar los módulos, decidiendo practicar con una petición de tipo "post" para enviar dos parámetros, lo que implicaba el uso del módulo 'search'.
- **Demostración de la petición de tipo Post utilizando Postman:** Se mostró cómo configurar una petición de tipo "post" en Postman, con la información viajando dentro del cuerpo ("body") de la petición, a diferencia de las peticiones "get". La meta era actualizar un registro de NetSuite usando el módulo 'record' y 'submit field', pidiendo que el parámetro incluyera forzosamente los atributos 'type', 'ID' y 'value'.
- **Implementación y prueba de la actualización de registros:** Se demostró que la petición de tipo "post" podía actualizar registros en NetSuite, como la fecha de nacimiento de un cliente, utilizando un código genérico para facilitar la práctica. Se configuró la respuesta para incluir el 'ID' de confirmación y lo que se había procesado, verificando la actualización del cliente en NetSuite. Se observó que el desarrollo del 'restlet' era similar a la práctica del 'suitelet', pero con el uso de un formulario visual.
- **Manejo de errores y validación de datos en las peticiones:** Se probó el envío de datos no válidos (por ejemplo, un 'ID' 55 o un año 3000), lo que generó errores

que se capturaron y se incluyeron en el mensaje de respuesta para indicar por qué no se había realizado la actualización. La limitación del valor 3000 se debió a una configuración previa del campo que restringía el rango máximo y mínimo. Se concluyó que el desarrollo del 'post' había quedado listo para ser guardado y registrado con un 'commit'.

- **Evaluación de la práctica y decisión sobre la herramienta Postman:** Iztac Amaya preguntó si la demostración había sido fácil de entender, a lo que Jorge Nicolas Moctezuma Arellano respondió que era comprensible, aunque la práctica era necesaria. Se recomendó a los participantes instalar Postman para familiarizarse con él, ya que probablemente lo utilizarían en futuros proyectos y facilitaría la simulación de terceros en integraciones.
- **Asignación de la práctica final y revisión de dudas:** Se decidió que la práctica se centraría únicamente en el desarrollo de un 'suitelet', excluyendo el 'restlet', para que los participantes pudieran practicar con los módulos 'record' y 'search' de NetSuite. La práctica consistiría en generar una pantalla visual con dos campos (selección de cliente y campo a actualizar) que, al presionar un botón, realizara la actualización del registro, similar a lo que se había hecho con el 'restlet'.
- **Detalles logísticos y cierre de la sesión:** Se confirmó que para acceder a los videos se debía solicitar acceso a través de la cuenta de Hostinger. No hubo más dudas sobre el material. La práctica debía realizarse en la misma rama de Git utilizada para el 'user event script' y el 'client script', con el objetivo de que todos los participantes tuvieran estos tres 'scripts' funcionales, actualizados y en modo de prueba en NetSuite .

Pasos siguientes recomendados

- ☐ Jorge Nicolas Moctezuma Arellano trabajará y desarrollará el client script que no pudo ver en NetSuite.
- ☐ Iztac Amaya revisará con Ollin Palafox sus problemas de manifiesto en Visual Code y la forma de poner scripts en testing/producción.
- ☐ Iztac Amaya programará un mini curso o explicación de Git para ayudar a Jorge Nicolas Moctezuma Arellano con sus conflictos.
- ☐ The group debe buscar normalizarse con Git, el proyecto SDF, Visual Code, NetSuite y familiarizarse con el proceso de subir y descargar scripts.

- ☐ The group practicar4 los ejercicios de los scripts setlet y restlet que Iztac Amaya dejar4.

Revisa las notas de Gemini para asegurarte de que sean correctas. [Obt4n consejos y descubre c4mo toma notas Gemini](#)

Danos tu opini4n sobre el uso de Gemini para tomar notas en una [breve encuesta](#).